

Plan d'Implémentation Technique : Plateforme Shop-ia

1. Fondations de l'Infrastructure et Configuration Core (Supabase)

L'architecture de Shop-ia repose sur une approche **Backend-as-a-Service (BaaS)** exploitant l'écosystème **Supabase (v2.98.0)**. Ce choix technique est dicté par la nécessité d'assurer une scalabilité horizontale immédiate et une synchronisation des stocks en temps réel via WebSockets, un impératif critique pour la gestion de produits périssables et artisanaux à disponibilité limitée. En déportant la gestion de l'infrastructure vers une solution gérée, l'ingénierie se concentre sur la logique métier complexe tout en garantissant une haute disponibilité native.

Initialisation et Prérequis

L'environnement local doit impérativement respecter les versions suivantes pour garantir la compatibilité des dépendances : **Node.js >= 18.0** et **npm >= 9.0**. Les étapes d'initialisation de la couche de persistance sont les suivantes :

- **Exécution du script `init_database.sql`** : Définition du schéma relationnel, des fonctions de base et de l'orchestration initiale des tables.
- **Activation de l'extension `vector`** : Prérequis indispensable pour transformer PostgreSQL en base de données vectorielle, supportant ainsi les calculs de similarité nécessaires à la recherche sémantique.
- **Application des schémas d'embeddings** : Bien qu'indiqués comme optionnels dans la documentation de base, les scripts `apply_vectors_part1.sql` (et suivants) doivent être exécutés pour indexer le catalogue et activer les fonctionnalités d'IA avancées.

Configuration de l'Authentification et Connectivité

La couche d'authentification utilise le fournisseur d'identité interne de Supabase via email/mot de passe. La liaison frontend-backend est assurée par l'injection de variables d'environnement strictes : | Variable | Fonction Opérationnelle || ----- | ----- ||
VITE_SUPABASE_URL | Point d'entrée de l'API REST et client WebSocket pour le Realtime. || VITE_SUPABASE_ANON_KEY | Clé publique autorisant l'accès aux données exposées via les politiques RLS. |

Une fois cette infrastructure saine et les variables de connexion stabilisées, la plateforme dispose du socle de données nécessaire pour intégrer la couche d'intelligence artificielle.

2. Intégration de l'Écosystème Intelligence Artificielle

L'avantage concurrentiel de Shop-ia ne réside pas seulement dans son catalogue, mais dans la réduction de la distance sémantique entre l'intention de l'utilisateur et la découverte de produits complexes (accords mets/vins, saisonnalité). L'Assistant Shopia et la recherche vectorielle transforment la navigation transactionnelle en une expérience de conseil assistée par LLM.

Configuration OpenRouter et Recherche Vectorielle

La recherche sémantique s'appuie sur le modèle `openai/text-embedding-3-small` via le proxy OpenRouter.

- **Optimisation Vectorielle** : Nous avons configuré les vecteurs sur **768 dimensions** . Ce choix architectural est une optimisation spécifique visant à équilibrer la précision de la recherche sémantique et les performances de stockage/indexation au sein de Supabase, tout en assurant une compatibilité avec le schéma de données défini.
- **Variables obligatoires** : La clé VITE_OPENROUTER_API_KEY est strictement requise pour l'initialisation du client d'embedding.

Déploiement de Shopia Assistant (Gemini)

L'intelligence conversationnelle repose sur **Google GenAI (v1.29.0)** via l'API Gemini. Cet agent gère les flux de conseils personnalisés et le traitement des quiz de préférences clients.**Checklist de validation IA :**

- Injection de VITE_OPENROUTER_API_KEY (Mandatoire pour les embeddings).
- Configuration de VITE_OPENROUTER_EMBED_MODEL sur openai/text-embedding-3-small.
- Paramétrage de VITE_OPENROUTER_EMBED_DIMENSIONS à 768.
- Injection de VITE_GEMINI_API_KEY pour l'activation de l'assistant live.
- Test de latence sur les requêtes de recherche sémantique pour valider l'indexation. Une fois la pertinence des résultats de recherche et la fluidité de l'assistant validées, le flux logique de l'application converge naturellement vers la sécurisation du tunnel transactionnel.

3. Déploiement du Tunnel d'Achat et Paiements (Viva Wallet)

La fiabilité de l'encaissement est la clé de voûte de la confiance client. La conformité bancaire et la fluidité du checkout sont priorisées pour réduire les taux d'abandon de panier. L'intégration de **Viva Wallet** assure un traitement sécurisé conforme aux standards PCI-DSS.

Configuration du Connecteur Viva Wallet

L'intégration repose sur le paramétrage des credentials API et de l'URL d'environnement :

1. **Identifiants** : Utilisation de VITE_VIVA_CLIENT_ID et VITE_VIVA_CLIENT_SECRET.
2. **Environnement** : Définition de VITE_VIVA_WALLET_BASE_URL (URL de démo pour le bac à sable technique, URL de production pour le lancement).
3. **Atomicité** : Les mises à jour de stock post-paiement sont orchestrées via **Supabase Realtime** pour prévenir tout risque de "race condition" (concurrence d'accès) sur les produits à faible stock.

Gestion des Flux Transactionnels et Fidélité

Afin de garantir l'intégrité des données, la logique post-achat ne doit pas reposer uniquement sur le frontend :

- **Triggers de Base de Données** : L'incrémentation des points de fidélité et la création de l'historique de parrainage doivent être traitées par des **Database Functions/Triggers** dans Supabase. Cela garantit une mise à jour atomique et immuable, indépendamment de la session client.
- **Synchronisation** : Les statuts de commande passent de "En attente" à "Payée" dès réception du webhook de confirmation Viva Wallet. La réussite de ces transactions

critiques impose une gouvernance stricte de l'accès aux données sensibles des profils clients et des commandes.

4. Sécurisation Granulaire via les Politiques RLS (Row Level Security)

La souveraineté des données et l'isolation des tenants sont gérées par le mécanisme de **Row Level Security (RLS)**. Dans une base de données partagée, le RLS est l'unique rempart garantissant que les données des clients et des producteurs restent hermétiques.

Architecture de Sécurité Supabase

Les politiques définies dans `init_database.sql` s'appuient systématiquement sur la fonction `auth.uid()` pour filtrer les accès :

- **Panier Persistant et Favoris** : L'accès est restreint au propriétaire de la session (`user_id = auth.uid()`).
- **Dashboard Admin** : L'accès aux tables de gestion (Analytics, Stocks, Marketing) est réservé aux comptes disposant du rôle administrateur. Les imports massifs de données via le module "Import Producteurs" utilisent le service_role pour contourner temporairement les politiques RLS tout en maintenant l'intégrité du catalogue.
- **Historique des Commandes** : Lecture autorisée pour l'utilisateur concerné, accès complet pour les administrateurs pour le support client.

Gouvernance des Accès Multi-tenant

Cette architecture prévient les fuites de données transversales. Même en cas de compromission d'un token client, l'attaquant reste limité au périmètre défini par son uid. Cette sécurité granulaire est le prérequis final validant l'ouverture de la plateforme au public.

5. Orchestration du Déploiement et Mise en Production

Le déploiement continu (CD) est assuré par **Vercel**, garantissant une disponibilité maximale et une gestion simplifiée des variables d'environnement. Le stack technique est figé sur **React 19.0.0**, **TypeScript 5.8.2** et **Vite 6.2.0**.

Workflow de Déploiement et Build

Le processus suit une séquence de validation rigoureuse :

1. **Build de Production** : Commande `npm run build` via Vite pour l'optimisation des bundles.
2. **Synchronisation Vercel** : Toutes les variables `VITE_*` doivent être déclarées dans l'interface Vercel avant le premier déploiement pour éviter les erreurs d'initialisation au runtime.
3. **Monitoring** : Utilisation de **Recharts (v3.7.0)** dans le dashboard admin pour monitorer en temps réel les performances post-lancement.

Validation et Lancement (SMOKE Tests)

Avant l'ouverture du trafic, les tests de validation suivants sont obligatoires :| Test de Validation | Objectif | Résultat Attendu || ----- | ----- | ----- || **Auth & Session** | Persistance de la connexion | Maintien du panier après rafraîchissement. || **Semantic Search** | Précision des 768 dimensions | Résultats pertinents sur requêtes complexes. || **Real-time Sync** | Prévention des race conditions | Décrémentement immédiat du stock visible. || **Viva Wallet** |

Simulation de transaction (Demo) | Génération d'un statut "Payé" et trigger fidélité. || **Admin**
Analytics | Intégrité des graphiques | Visualisation correcte des données via Recharts. |
La maintenance évolutive s'appuiera sur la ROADMAP.md, notamment pour les futures
itérations sur les abonnements récurrents et l'enrichissement du catalogue par IA.